claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-adb1b659bd00a23d1.jsonl`
- SHA-256: `c5c2e1abaf5b73f5ce61cce22b6fd15155bb07c2bb83007815227648513dcf12`
- Source modified: `2026-06-12T10:46:13+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`

Transcript

1. user (2026-06-12T10:43:40.355Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:
- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [low] out==src guard is case-sensitive; on Windows a case-variant --out can overwrite the source (C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:670)
`os.path.abspath(out) == os.path.abspath(src)` is a case-sensitive string compare. On this repo's primary platform (Windows, case-insensitive filesystem), `--out 'D:\v\MATCH.MP4' --video-path 'D:\v\match.mp4'` passes the guard, then ffmpeg -y opens the source file as its own output and destroys the original ? the one file the whole contract says is the system of record. Low likelihood (requires an explicit --out aimed at the source), but the guard exists precisely for this fat-finger and is broken on the platform in use.
Proposed fix: Compare os.path.normcase(os.path.realpath(...)) on both sides, and additionally use os.path.samefile(out, src) when `out` exists (also catches hardlinks/subst drives).

2. user (2026-06-12T10:43:47.104Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
* feat/prep-annotation-proxy

3. user (2026-06-12T10:43:48.744Z)

```
200-        print(f"  - Sport: {sport_str}")
201-        print(f"  - Commercial Use Allowed: {is_commercial} (License:
```

```
{license_type.value})")
202-        print(f"  - Registered {count} annotated segments.\n")
203-
204-        # Per-video observability report (next to the video). Never raises.
205-        from src.reporting import emit_import_report
206-        src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
207-        report_paths = emit_import_report(
208-            video_meta_obj=video_meta, annotations=annotations, is_valid=True,
209-            errors=[], count=count, sport=sport_str, source_format=src_fmt,
210:            fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
211-        )
212-        if report_paths:
213-            print(f"  - Observability report: {report_paths.get('technical_md')}")
214-            print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")
215-
216-    def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
217-        """Parses simple JSON or CSV annotations files."""
218-        parsed_items = []
219-        ext = os.path.splitext(filepath)[1].lower()
220-
--
660-        src = args.video_path
661-        if not os.path.exists(src):
662-            print(f"Error: video '{src}' not found.")
663-            sys.exit(1)
664-
665-        stem = os.path.basename(src).split('.')[0]
666-        # Mirror import-local's video_id derivation so the manifest row and a
667-        # later DB registration of the same source agree on the key.
668-        video_id = args.video_id or stem.lower().replace(" ", "_")
669-        out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670:        if os.path.abspath(out) == os.path.abspath(src):
671-            print("Error: --out must differ from --video-path.")
672-            sys.exit(1)
673-        if os.path.exists(out) and not args.force:
674-            print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675-            sys.exit(1)
676-
677-        try:
678-            src_info = proxylib.probe_video(src)
679-        except proxylib.ProxyError as e:
680-            print(f"Error: {e}")
--
685-            sys.exit(1)
686-        if proxylib.is_vfr(src_info) and not args.allow_vfr:
687-            print(f"Error: '{src}' looks variable-frame-rate "
688-                  f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
689-            print("  The vendor pipeline recomputes times as frame/fps, which is only "
690-                  "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691-                  "proceed anyway (timestamps stay identical to the original, but "
692-                  "frame/fps seconds are approximate for BOTH files).")
693-            sys.exit(1)
694-
695:        os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696-        cmd = proxylib.build_proxy_command(src, out, height=args.height,
697-                                           crf=args.crf, gop=args.gop,
698-                                           preset=args.preset)
699-        print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700-              f"fps {src_info.avg_fps:g} preserved)...")
701-        try:
702-            proxylib.run_ffmpeg(cmd)
703-            proxy_info = proxylib.probe_video(out)
704-        except proxylib.ProxyError as e:
```

```
705-            print(f"Error: {e}")
--
719-                      f"share it with the vendor.")
720-            sys.exit(1)
721-
722-        print("Hashing source + proxy (provenance)...")
723-        source_md5 = proxylib.md5_file(src)
724-        proxy_md5 = proxylib.md5_file(out)
725-
726-        from datetime import datetime, timezone
727-        proxylib.append_manifest_row(args.manifest, {
728-            "video_id": video_id,
729:            "source_path": os.path.abspath(src),
730-            "source_md5": source_md5,
731:            "proxy_path": os.path.abspath(out),
732-            "proxy_md5": proxy_md5,
733-            "fps": f"{src_info.avg_fps:g}",
734-            "frame_count": src_info.nb_frames,
735-            "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None else
""",
736-            "source_resolution": src_info.resolution,
737-            "proxy_resolution": proxy_info.resolution,
738-            "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
739-                                 f"preset={args.preset} height<={args.height} "
740-                                 f"fps_mode=passthrough audio=aac160k"),
741-            "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
--
784-                  f"renamed (boxes without a matched rally_number are skipped).")
785-            sys.exit(1)
786-
787-        write_canonical_csv(result.records, args.out)
788-
789-        # Observability report next to the canonical CSV. Never raises.
790-        from src.reporting import emit_delivery_report
791-        src_name = os.path.splitext(os.path.basename(args.input))[0]
792-        dur = (result.meta.get("stop_frame") / result.fps) if (result.meta.get("stop_frame")
and result.fps) else None
793-        emit_delivery_report(
794:            out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
795-            records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
796-            video_duration_s=dur, source_format="cvat", step=result.step,
797-        )
798-
799-        report = render_issue_report(result, os.path.basename(args.input))
800-        if args.issues:
801-            with open(args.issues, "w", encoding="utf-8") as f:
802-                f.write(report)
803-        print(report)
804-        print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
--
1048-
1049-         if start_time is None or end_time is None:
1050-             print("Segment not found or invalid input.")
1051-             return
1052-
1053-         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
1054-
1055-         # Check video location
1056-         play_url = None
1057-         if video.local_path and os.path.exists(video.local_path):
1058:             abs_path = os.path.abspath(video.local_path)
1059-             # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
1060-             play_url = f"file:///{abs_path.replace(os.sep, '/')}#t={start_time},{end_time}"
```

```
1061-        elif video.video_url:
1062-            # YouTube URL: seek to start time
1063-            if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
1064-                t_sec = int(start_time)
1065-                sep = "&" if "?" in video.video_url else "?"
1066-                play_url = f"{video.video_url}{sep}t={t_sec}s"
1067-            else:
1068-                play_url = video.video_url
```

4. user (2026-06-12T10:44:21.110Z)

```
640             if skipped_noncomm:
641                 print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
642             if skipped_empty:
643                 print(f"Skipped (no annotations) : {skipped_empty}")
644             print(f"Manifest        : {manifest_path}\n")
645 
646         def prep_annotation_proxy(self, args):
647             """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648 
649             The proxy is what the annotation vendor receives: downscaled and
650             scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651             but with the source's EXACT frame count / fps / timeline, so frame N
652             on the proxy is frame N on the original and returned labels apply to
653             the original unchanged. Parity is verified after the encode; a proxy
654             that fails verification is deleted, never shared. Provenance (source +
655             proxy MD5s) is appended to the proxy manifest ? downstream processing
656             (import-local, export, the indexer) keeps running on the ORIGINAL file.
657             """
658             from src.core import proxy as proxylib
659 
660             src = args.video_path
661             if not os.path.exists(src):
662                 print(f"Error: video '{src}' not found.")
663                 sys.exit(1)
664 
665             stem = os.path.basename(src).split('.')[0]
666             # Mirror import-local's video_id derivation so the manifest row and a
667             # later DB registration of the same source agree on the key.
668             video_id = args.video_id or stem.lower().replace(" ", "_")
669             out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670             if os.path.abspath(out) == os.path.abspath(src):
671                 print("Error: --out must differ from --video-path.")
672                 sys.exit(1)
673             if os.path.exists(out) and not args.force:
674                 print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675                 sys.exit(1)
676 
677             try:
678                 src_info = proxylib.probe_video(src)
679             except proxylib.ProxyError as e:
680                 print(f"Error: {e}")
681                 sys.exit(1)
682             if src_info.avg_fps is None:
683                 print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684                       f"contract needs it.")
685                 sys.exit(1)
686             if proxylib.is_vfr(src_info) and not args.allow_vfr:
687                 print(f"Error: '{src}' looks variable-frame-rate "
688                       f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
689                 print("  The vendor pipeline recomputes times as frame/fps, which is only "
690                       "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691                       "proceed anyway (timestamps stay identical to the original, but "
692                       "frame/fps seconds are approximate for BOTH files).")
```

```
                sys.exit(1)

            os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
            cmd = proxylib.build_proxy_command(src, out, height=args.height,
                                               crf=args.crf, gop=args.gop,
                                               preset=args.preset)
            print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
                  f"fps {src_info.avg_fps:g} preserved)...")
            try:
                proxylib.run_ffmpeg(cmd)
                proxy_info = proxylib.probe_video(out)
            except proxylib.ProxyError as e:
                print(f"Error: {e}")
                sys.exit(1)

            errors = proxylib.verify_parity(src_info, proxy_info)
            if errors:
                print("Error: proxy FAILED frame<->time parity verification:")
                for err in errors:
                    print(f"  - {err}")
                try:
                    os.remove(out)
                    print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
                          f"poisons golden labels).")
                except OSError:
                    print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
                          f"share it with the vendor.")
                sys.exit(1)

            print("Hashing source + proxy (provenance)...")
            source_md5 = proxylib.md5_file(src)
            proxy_md5 = proxylib.md5_file(out)

            from datetime import datetime, timezone
            proxylib.append_manifest_row(args.manifest, {
                "video_id": video_id,
                "source_path": os.path.abspath(src),
                "source_md5": source_md5,
                "proxy_path": os.path.abspath(out),
                "proxy_md5": proxy_md5,
                "fps": f"{src_info.avg_fps:g}",
                "frame_count": src_info.nb_frames,
                "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
else "",
                "source_resolution": src_info.resolution,
                "proxy_resolution": proxy_info.resolution,
                "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
                                     f"preset={args.preset} height<={args.height} "
                                     f"fps_mode=passthrough audio=aac160k"),
                "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
            })

            src_mb = os.path.getsize(src) / (1024 * 1024)
            out_mb = os.path.getsize(out) / (1024 * 1024)
            print("\n=== Annotation proxy ready ===")
            print(f"  Video ID    : {video_id}")
            print(f"  Source      : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
            print(f"  Proxy       : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")
            print(f"  Parity      : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
            print(f"  Manifest    : {args.manifest}")
            print("\nNext:")
            print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
```

```
754                   "box of every rally on the exact contact/dead frame).")
755             print("  2. On delivery, convert-cvat may use --video-path with either file "
756                   "(identical fps/frame count by construction).")
757             print("  3. import-local / export keep pointing at the ORIGINAL file ? the "
758                   "indexer keys videos by its MD5.\n")
759
```

5. user (2026-06-12T10:44:22.262Z)

```
150-    """
151-    if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152-        return True
153-    return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154-
155-
156:def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157-                       crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158-                       preset: str = DEFAULT_PRESET) -> List[str]:
159-    """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160-
161-    Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162-    trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163-    1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164-    never upscales. Audio is kept (contact sound is the annotator's best
165-    boundary cue).
166-    """
167-    return [
168:        "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169-        "-i", src,
170-        "-map", "0:v:0", "-map", "0:a?",
171-        "-vf", f"scale=-2:min({height}\\,ih)",
172-        "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173-        "-g", str(gop), "-pix_fmt", "yuv420p",
174-        "-fps_mode", "passthrough",
175-        "-c:a", "aac", "-b:a", "160k",
176-        "-movflags", "+faststart",
177-        dst,
178-    ]
179-
180-
181:def run_ffmpeg(cmd: List[str]) -> None:
182-    """Run an ffmpeg command, streaming its progress to the console."""
183-    if not shutil.which("ffmpeg"):
184-        raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185-    try:
186-        # No capture: -stats progress and any warnings go straight to the user.
187-        subprocess.run(cmd, check=True)
```

6. user (2026-06-12T10:44:50.765Z)

```
abspath equal       : False
exists(case-variant): True
normcase+realpath eq: True
samefile            : True
```

7. user (2026-06-12T10:45:10.895Z)

```
--- source created:
8448
[out#0/mp4 @ 0000026a3f14dcc0] Codec AVOption b:a (set bitrate (in bits/s)) has not been used
for any stream. The most likely reason is either wrong type (e.g. a video option with no video
streams) or that it is a private option of some decoder which was not actually used for any
stream.
frame=   30 fps=0.0 q=-1.0 Lsize=       8KiB time=00:00:00.93 bitrate=  71.9kbits/s speed=33.4x
```

```
elapsed=0:00:00.02
--- ffmpeg exit: 0
--- file after:
8385
[FORMAT]
duration=1.000000
[/FORMAT]
```

8. user (2026-06-12T10:45:41.475Z)

```
77-                  f"(often a fractional id like 7.5 truncating to 7).")
78-            print("  -> re-run with --normalize, or normalize first: "
79:                  "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
80-            sys.exit(1)
81-
--
90-                print(f"Error: video_id '{video_id}' already has {n} annotated segment(s); "
91-                      f"importing would REPLACE them.")
92:                print("  -> pass --force to overwrite, or use a distinct --video-id to keep
both.")
93-                sys.exit(1)
94-            if n > 0:
95:                print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
96-
97-    def import_local(self, args):
--
407-        `resolution`, and `fps` are refreshed. By default it skips videos that
408-        already have a local file at or above `--min-height`, so re-runs are cheap
409:        and resumable; `--force` re-fetches everything.
410-        """
411-        storage = self.config.get("video_storage", {})
--
454-            dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
455-
456:            # Skip if we already have a good-enough local file (unless --force).
457-            if not args.force and os.path.exists(dest_path):
458-                _, height, _ = probe_resolution_fps(dest_path)
--
669-        out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670-        if os.path.abspath(out) == os.path.abspath(src):
671:            print("Error: --out must differ from --video-path.")
672-            sys.exit(1)
673-        if os.path.exists(out) and not args.force:
674:            print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675-            sys.exit(1)
676-
--
815-
816-        Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
817:        CSV (--csv). Writes, into --out-dir:
818-          <id>_report.md    human report (scoreboard + issues + rallies to review)

[Showing results with pagination = limit: 40]
```

9. user (2026-06-12T10:46:07.458Z)

```
Structured output provided successfully
```

10. assistant (2026-06-12T10:46:12.938Z)

```
Verification complete. The claim is real at low severity: the line-670 abspath comparison is
case-sensitive on Windows, ffmpeg's own same-file check (a strcmp) also misses the case variant,
and a live reproduction on this machine showed the source file being silently overwritten in
```

place with exit 0 ? though the `--force` requirement at line 673 (which the claim omits) keeps the trigger a rare double fat-finger.